

Kiste v0.9.1 Spec

Standard Config, Output, Command, and Split-Ready Architecture Contract

One-sentence spec: Kiste v0.9.1 standardizes the Kiste project contract by defining kiste.yaml, kiste.lock, .kiste/ outputs, a clean command tree, and a split-ready architecture where kiste-py and kiste-cli both depend on kiste-core but do not depend on each other.

1. Release Goal

v0.9.1 is a cleanup and standardization release. It should not add major new features. Its purpose is to make Kiste boring, predictable, and ready for users and contributors.

- Standard config file: kiste.yaml.
- Standard lockfile: kiste.lock.
- Standard generated output directory: .kiste/.
- Standard command model with duplicate commands removed.
- Standard architecture boundary for future split into kiste-core, kiste-py, and kiste-cli.

2. Standard Files

File / directory	Owner	Purpose
kiste.yaml	Human + Kiste assisted	Main project/workspace contract. Describes what the repo is and what Kiste should know.
kiste.lock	Kiste generated	Freezes Kiste, schema, plugin API, and module versions for reproducibility.
.kiste/	Kiste generated	Stores scan results, reports, plans, graphs, manifests, cache, and temporary state.

Rule: kiste.yaml describes what the project is. kiste.lock freezes how Kiste interpreted it. .kiste/ records what Kiste found, planned, or generated.

3. Standard Config File: kiste.yaml

The main user-facing file is kiste.yaml. It is similar in role to package.json for Node projects or a Dockerfile for container builds: it gives the project a predictable contract that tools can inspect.

```
kiste: "0.9.1"
schema: "kiste.config/v0.9.1"
kind: Project

project:
  name: robot-api
  type: service
  description: Robot API backend

modules:
  enabled:
    - repo
    - tokens
    - workspace
    - gitops
    - secrets

runtime:
  language: python
  framework: fastapi
  entrypoint: main:app
  start: uvicorn main:app --host 0.0.0.0 --port 8000

container:
  dockerfile: Dockerfile
  image: docker.io/kistebox/robot-api:latest
  ports:
    - 8000
```

```

env:
  required:
    - DATABASE_URL
    - SECRET_KEY
  optional:
    - LOG_LEVEL

target:
  preferred: kubernetes
  alternatives:
    - cloud-run
    - azure-container-apps

bootstrap:
  plan_before_apply: true
  write_requires_confirm: true

```

Allowed top-level sections

```

kiste:
schema:
kind:
project:
modules:
workspace:
runtime:
services:
container:
dependencies:
env:
secrets:
tokens:
images:
data:
blob:
gitops:
argocd:
cloud:
target:
bootstrap:
release:

```

Unknown top-level sections should fail validation unless explicitly allowed through an extension mechanism. This keeps the config stable and prevents silent mistakes.

4. Standard Lockfile: kiste.lock

The lockfile records the exact Kiste and module versions used to interpret a project. It is the foundation for future package-manager-like behavior, but v0.9.1 should only implement locking and verification, not a marketplace.

```

{
  "kiste": "0.9.1",
  "schema": "kiste.config/v0.9.1",
  "plugin_api": "0.1",
  "generated_at": "2026-05-30T00:00:00Z",
  "modules": {
    "repo": "0.9.1",
    "tokens": "0.9.1",
    "workspace": "0.9.1",
    "gitops": "0.9.1",
    "secrets": "0.9.1",
    "data": "0.9.1",
    "blob": "0.9.1"
  }
}

```

5. Standard Output Layout: .kiste/

.kiste/ is Kiste-generated state. It should contain outputs and reports, not user-authored source truth. Some files may be committed, but cache and local-only data should not be committed.

```

.kiste/
  spec.json
  graph.json
  modules.json
  doctor-report.json

```

```

bootstrap-plan.json
target-plan.json
release-check.json

blobs/
  weld-detection-export.manifest.json

gitops/
  gitops-plan.json

argocd/
  argocd-plan.json

cloud/
  cloud-plan.json

self/
  self-check.json
  self-review.md
  release-plan.md

cache/
tmp/
logs/

```

Commit policy

Commit	Usually commit	Do not commit
kiste.yaml kiste.lock	.kiste/blobs/*.manifest.json .kiste/release-check.json if useful for CI	.kiste/cache/ .kiste/tmp/ .kiste/logs/ .kiste/*.local.json

6. Standard Command Model

v0.9.1 removes duplicate commands and assigns one canonical command per job. The command names should match the user mental model.

Command group	Canonical commands	Meaning
Core	kiste init kiste doctor kiste scan .	init creates minimal config; doctor diagnoses health; scan inspects a repo and pro
Config	kiste config validate kiste config print kiste config schema kiste config migrate	Manage and validate kiste.yaml.
Bootstrap	kiste bootstrap plan kiste bootstrap apply --confirm kiste bootstrap validate kiste bootstrap status	Bootstrap is a staged process for repo changes Kiste wants to make.
Target	kiste target plan	Decide where/how the app should run.
GitOps	kiste gitops generate kiste gitops validate	Generate/validate deployment repo structure.
Argo CD	kiste argocd generate kiste argocd validate kiste argocd plan	Generate/validate Argo CD control manifests.
Cloud	kiste cloud plan kiste cloud check kiste cloud credentials check kiste cloud cluster spec	Infrastructure planning only; no cloud apply in v0.9.1.

Data	kiste data inspect kiste data scan kiste data check kiste data card kiste data export-plan	Logical dataset meaning and readiness.
Blob	kiste blob inspect kiste blob scan kiste blob manifest kiste blob check kiste blob sample kiste blob export-plan	Physical storage reference, metadata-first manifests, and safe sampling.
Module	kiste module list kiste module graph kiste module doctor kiste module lock --write kiste module verify	Static plugin/module state and lock verification.
Maintainer	kiste architecture check kiste release check kiste self check	Boundary checks, release readiness, and Kiste dogfooding.

Commands to remove or alias

Old / duplicate	Decision	Reason
kiste check	Alias to kiste doctor or remove	doctor is the canonical health command.
kiste bootstrap report	Alias to kiste bootstrap status or remove	plan/status already cover report behavior.
many bootstrap subcommands such as bootstrap with bootstrap plan apply --level	Bootstrap with bootstrap plan apply	.Bootstrap is a process, not many unrelated actions.

7. Bootstrap vs Target

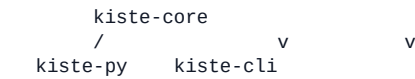
Bootstrap and target are different concepts and should not be mixed.

Concept	Question it answers	Example command	Output
target	Where/how should this app run?	kiste target plan	Kubernetes, Cloud Run, Lambda container, Azure
bootstrap	What repo changes are needed to prepare for that destination?	kiste bootstrap plan	Create/update kiste.yaml, kiste.lock, .kiste/module

Rule: target chooses the deployment destination. bootstrap prepares the repo for that destination.

8. Standard Architecture Boundary

v0.9.1 should make Kiste split-ready. The future split is kiste-core, kiste-py, and kiste-cli. The critical rule is that kiste-py and kiste-cli are independent sibling interfaces over kiste-core.



Do not use this dependency shape:

kiste-cli -> kiste-py -> kiste-core

And never allow:

```
kiste-py -> kiste-cli
kiste-core -> kiste-cli
kiste-core -> kiste-py
```

Package responsibilities

Package	Owns	Must not own
kiste-core	Models, schemas, config parser, scanner engine, workspace graph, plugin/module registry, file reports, bootstrap plan	Terminal UX, kiste lock file, report objects, bootstrap plan

kiste-py	Python-friendly wrapper functions such as scan_repo(), load_config(), run_doctor(), plugin_bootstrap(), plan_target().	Code of conduct, plugin bootstrap, help plan.
kiste-cli	Argument parsing, help text, terminal rendering, JSON output, exit codes, interactive documentation.	Command line interface, dependency on kiste-py.

Example wrapper design

```
# kiste-py/src/kiste/scanner.py
from kiste_core.scanner import scan_repo as _scan_repo

def scan_repo(path: str = "."):
    return _scan_repo(path)

# kiste-cli/src/kiste_cli/commands/doctor.py
from kiste_core.doctor import run_doctor
from kiste_cli.render import render_doctor_report

def doctor_command(path: str = ".", json_output: bool = False):
    report = run_doctor(path)
    if json_output:
        print(report.model_dump_json(indent=2))
    else:
        render_doctor_report(report)
```

9. Standard Monorepo Layout

Use a monorepo now, but keep boundaries strict enough that the project can split later without architectural surgery.

```
kiste/
  pyproject.toml
  README.md

  packages/
    kiste-core/
      pyproject.toml
      src/kiste_core/

    kiste-py/
      pyproject.toml
      src/kiste/

  apps/
    kiste-cli/
      pyproject.toml
      src/kiste_cli/

  plugins/
    kiste-repo/
    kiste-workspace/
    kiste-gitops/
    kiste-data/
    kiste-blob/
    kiste-cloud/
    kiste-argocd/

  contracts/
    schemas/
      kiste-config-v0.9.1.schema.json
      repo-spec.schema.json
      workspace-spec.schema.json
      dataset-spec.schema.json
      blob-ref.schema.json

  tests/
    core/
    py/
    cli/
    plugins/
    architecture/
```

10. Architecture Tests

Architecture tests enforce the boundary so the split-ready design stays real.

```
tests/architecture/
  test_core_imports_no_wrapper_or_cli.py
  test_py_imports_no_cli.py
```

```

test_cli_imports_no_py.py
test_core_contracts_exist.py
test_public_config_schema.py
test_standard_output_layout.py

from pathlib import Path

FORBIDDEN = {
    "packages/kiste-core": ["kiste_cli", "from kiste ", "import kiste."],
    "packages/kiste-py": ["kiste_cli"],
    "apps/kiste-cli": ["from kiste ", "import kiste."],
}

def test_dependency_boundaries():
    for root, forbidden_imports in FORBIDDEN.items():
        for path in Path(root).rglob("*.py"):
            text = path.read_text(encoding="utf-8")
            for forbidden in forbidden_imports:
                assert forbidden not in text, f"{path} illegally imports {forbidden}"

```

11. Validation Rules

kiste config validate should check the project contract before any generated output is trusted.

- [OK] valid schema version.
- [OK] known kind: Project, Workspace, Plugin, or Template.
- [OK] project name exists.
- [OK] enabled modules are known.
- [OK] service names are unique.
- [OK] token secret_reference uses env:// or approved secret backend.
- [OK] blob refs do not download by default.
- [OK] cloud mode is plan-only unless admin mode is enabled.
- [OK] Argo CD prune is false by default.

Failures should include plaintext tokens, unknown top-level sections, duplicate service names, unsafe blob download defaults, or cloud apply mode without admin mode.

12. What v0.9.1 Should Not Do

- No new cloud providers.
- No new data providers.
- No remote plugin install.
- No automatic deployment.
- No automatic Argo CD sync.
- No automatic large downloads.
- No auto secret rotation.
- No marketplace.

v0.9.1 only standardizes config, output layout, command meaning, architecture boundary, validation rules, and migration path.

13. Acceptance Criteria

v0.9.1 is complete when these commands work and produce the standardized files and reports:

```

kiste init
kiste config validate
kiste config print

```

```
kiste config schema
kiste bootstrap plan
kiste bootstrap apply --confirm
kiste bootstrap validate
kiste doctor
kiste scan .
kiste target plan
kiste module lock --write
kiste module verify
kiste architecture check
```

Expected outputs:

- kiste.yaml created and validated.
- kiste.lock written and verified.
- .kiste/spec.json generated by scan.
- .kiste/doctor-report.json generated by doctor.
- .kiste/bootstrap-plan.json generated by bootstrap plan.
- .kiste/target-plan.json generated by target plan.
- kiste-core, kiste-py, and kiste-cli dependency boundaries verified.
- No duplicate canonical commands remain.

14. Final Rule

Kiste v0.9.1 should make the external project contract and internal architecture boring, stable, and split-ready.

Final contract: kiste.yaml is the config, kiste.lock is the lock, .kiste/ is the output, commands are canonical, and kiste-py plus kiste-cli are independent sibling interfaces over kiste-core.